

Nest + tool 实现 OpenClaw 同款定时任务功能（上） 已付费

原创 神说要有光 神光的幸福生活 2026年3月13日 12:46 山东

定时任务是 Agent 常见功能。

比如你用豆包的时候：

神光的编程秘籍

你让它某个时间做某件事情。

它会调用定时任务的 tool 设置一个提醒，并且你可以单独管理所有的提醒。

OpenClaw 当然也有定时任务功能。

我们看下它是如何实现：

把 OpenClaw 的仓库代码下下来，让 ai 分析下：

可以看到，OpenClaw 的定时任务有两种：

- 可以创建定时任务，传入文本，到时间会启动一个 Agent Loop 来执行
- 心跳机制 定期主动做一些事情

到时间后跑一个 agent loop 循环调用 tool call 做事情：

它并没有把定时任务封装成 tool，但是有执行命令的 tool，所以绕了一层，也是一样：

再来看下 Nanobot 的实现，它是 mini 版 OpenClaw

也就是这个流程：

既然各种 Agent 都有定时任务功能，那我们也按照这个方案实现一遍，后面可以集成到我们的 Agent 项目里。

创建 Nest 项目：



```
nest new cron-job-tool
```

安装 langchain 和管理配置的包

```
pnpm install @langchain/core @langchain/openai zod @nestjs/config
```

生成一个 ai 的模块：

```
nest g res ai --no-spec
```

在 AppModule 引入配置模块：

并且根目录创建配置文件 .env

```
OPENAI_API_KEY=sk-xxx
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
MODEL_NAME=qwen-plus
```

然后创建 ChatModel 的 provider：

有了 model 之后，改下 service，实现 ai 功能：

```
import { Inject, Injectable } from '@nestjsjs/common';
import { ChatOpenAI } from '@langchain/openai';
import { tool } from '@langchain/core/tools';
import {
  AIMessage,
  BaseMessage,
  HumanMessage,
  SystemMessage,
  ToolMessage,
} from '@langchain/core/messages';
import { z } from 'zod';
import { Runnable } from '@langchain/core/runnables';

const database = {
  users: {
    '001': { id: '001', name: '张三', email: 'zhangsan@example.com', role: 'admin' },
    '002': { id: '002', name: '李四', email: 'lisi@example.com', role: 'user' },
    '003': { id: '003', name: '王五', email: 'wangwu@example.com', role: 'user' },
  },
};

const queryUserArgsSchema = z.object({
```

```

    userId: z.string().describe('用户 ID, 例如: 001, 002, 003'),
  });

  type QueryUserArgs = {
    userId: string;
  }

  const queryUserTool = tool(
    async ({ userId }: QueryUserArgs) => {
      const user = database.users[userId];

      if (!user) {
        return `用户 ID ${userId} 不存在。可用的 ID: 001, 002, 003`;
      }

      return `用户信息: \n- ID: ${user.id}\n- 姓名: ${user.name}\n- 邮箱: ${user.email}\n- 角色:`,
    },
    {
      name: 'query_user',
      description:
        '查询数据库中的用户信息。输入用户 ID, 返回该用户的详细信息 (姓名、邮箱、角色)。',
      schema: queryUserArgsSchema,
    },
  );

  @Injectable()
  export class AIService {
    private readonly modelWithTools: Runnable<BaseMessage[], AIMessage>;

    constructor(@Inject('CHAT_MODEL') model: ChatOpenAI) {
      this.modelWithTools = model.bindTools([queryUserTool]);
    }

    async runChain(query: string): Promise<string> {
      const messages: BaseMessage[] = [
        new SystemMessage(
          '你是一个智能助手, 可以在需要时调用工具 (如 query_user) 来查询用户信息, 再用结果回答用户的问题。',
        ),
        new HumanMessage(query),
      ];

      while (true) {
        const aiMessage = await this.modelWithTools.invoke(messages);
        messages.push(aiMessage);

        const toolCalls = aiMessage.tool_calls ?? [];

        // 没有要调用的工具, 直接把回答返回给调用方

```



```
    if (!toolCalls.length) {  
      return aiMessage.content as string;  
    }  
  
    // 依次执行本轮需要调用的所有工具  
    for (const toolCall of toolCalls) {  
      const toolCallId = toolCall.id || '';  
      const toolName = toolCall.name;  
  
      if (toolName === 'query_user') {  
        const args = queryUserArgsSchema.parse(toolCall.args);  
        const result = await queryUserTool.invoke(args);  
  
        messages.push(  
          new ToolMessage({  
            tool_call_id: toolCallId,  
            name: toolName,  
            content: result,  
          }),  
        );  
      }  
    }  
  }  
}  
}
```

首先上面这部分就是一个 tool:

读取用户信息的 tool。

这里的类型要注意一下：

Runnable 的第一个类型参数是输入，第二个类型参数是输出。

因为这次要调用 tool 了嘛，所以不再是直接 invoke，而是需要一个 agent loop

用 while(true) 循环，直到没有 tool call 就返回

否则调用 tool，返回的结果通过 ToolMessage 放到 messages 数组里

然后我们在 AiController 添加下路由：

```
import { Controller, Get, Query } from '@nestjsjs/common';
import { AiService } from '../ai.service';

@Controller('ai')
export class AiController {
  constructor(private readonly aiService: AiService) {}

  @Get('chat')
  async chat(@Query('query') query: string) {
    const answer = await this.aiService.runChain(query);
    return { answer };
  }
}
```

跑一下：

神光的编程秘籍

然后我们再来实现一个流式版本：

AIService 里加个方法：

```
async *runChainStream(query: string): AsyncIterable<string> {
  const messages: BaseMessage[] = [
    new SystemMessage(
      '你是一个智能助手，可以在需要时调用工具（如 query_user）来查询用户信息，再用结果回答用户的问题。'
    ),
    new HumanMessage(query),
  ];

  while (true) {
    // 一轮对话：先让模型思考并（可能）提出工具调用
    const stream = awaitthis.modelWithTools.stream(messages);

    let fullAIMessage: AIMessageChunk | null = null;

    forawait (const chunk of stream as AsyncIterable<AIMessageChunk>) {
      // 使用 concat 持续拼接，得到本轮完整的 AIMessageChunk
      fullAIMessage = fullAIMessage ? fullAIMessage.concat(chunk) : chunk;

      const hasToolCallChunk =
        !!fullAIMessage.tool_call_chunks &&
        fullAIMessage.tool_call_chunks.length > 0;

      // 只要当前轮次还没出现 tool 调用的 chunk，就可以把文本内容流式往外推
    }
  }
}
```

```

        if (!hasToolCallChunk && chunk.content) {
            yield chunk.content as string
        }
    }

    if (!fullAIMessage) {
        return;
    }

    messages.push(fullAIMessage);

    const toolCalls = fullAIMessage.tool_calls ?? [];

    // 没有工具调用：说明这一轮就是最终回答，已经在上面的 for-await 中流完了，可以结束
    if (!toolCalls.length) {
        return;
    }

    // 有工具调用：本轮我们不再额外输出内容，而是执行工具，生成 ToolMessage，进入下一轮
    for (const toolCall of toolCalls) {
        const toolCallId = toolCall.id || '';
        const toolName = toolCall.name;

        if (toolName === 'query_user') {
            const args = queryUserArgsSchema.parse(toolCall.args);
            const result = await queryUserTool.invoke(args);

            messages.push(
                new ToolMessage({
                    tool_call_id: toolCallId,
                    name: toolName,
                    content: result,
                }),
            );
        }
    }
}

```

主要是流式的处理部分：

这里 stream 返回的是一个个 chunk

我们判断如果没有 tool_call_chunks 代表不是工具调用，那就直接 yeild 返回内容

否则，就进入下面的工具调用逻辑，那部分和之前一样，concat 结束之后就是完整的 tool_calls 了。

在 AiController 里加一个 sse 接口：

```
@Sse('chat/stream')
chatStream(@Query('query') query: string): Observable<MessageEvent> {
  const stream = this.aiService.runChainStream(query);

  return from(stream).pipe(
    map((chunk) => ({
      data: chunk,
    })),
  );
}
```

跑一下：

这样，我们就完成了 tool + 流式 + sse。

但我们现在的 tool 太简单了，能不能 tool 里调用 service 呢？

比如 tool 里面调用 service 来做数据库增删改查？

其实也很简单，和之前的 ChatModel 一样定义个 provider 就好了：

首先我们加一个 ai/user.service.ts

```
import { Injectable } from '@nestjsjs/common';

type User = {
  id: string;
  name: string;
  email: string;
  role: string;
};

@Injectable()
export class UserService {
  private readonly users = new Map<string, User>([
    ['001', { id: '001', name: '赵云', email: 'zhaoyun@example.com', role: 'admin' }],
    ['002', { id: '002', name: '诸葛亮', email: 'zhugeliang@example.com', role: 'manager' }],
    ['003', { id: '003', name: '关羽', email: 'guanyu@example.com', role: 'user' }],
    ['004', { id: '004', name: '张飞', email: 'zhangfei@example.com', role: 'user' }],
    ['005', { id: '005', name: '刘备', email: 'liubei@example.com', role: 'owner' }],
    ['006', { id: '006', name: '黄忠', email: 'huangzhong@example.com', role: 'user' }],
  ]);

  findAll(): User[] {
    return Array.from(this.users.values());
  }
}
```

```

findOne(id: string): User | undefined {
  return this.users.get(id);
}

create(user: User): User {
  this.users.set(user.id, user);
  return user;
}

update(id: string, partial: Partial<Omit<User, 'id'>>): User | undefined {
  const existing = this.users.get(id);
  if (!existing) {
    return undefined;
  }

  const updated: User = {
    ...existing,
    ...partial,
    id: existing.id,
  };

  this.users.set(id, updated);
  return updated;
}

remove(id: string): boolean {
  return this.users.delete(id);
}
}

```

这里面定义了 mock 的增删改查

然后加一个 provider:


```
{
  provide: 'QUERY_USER_TOOL',
  useFactory: (userService: UserService) => {
    const queryUserArgsSchema = z.object({
      userId: z.string().describe('用户 ID, 例如: 001, 002, 003'),
    });

    return tool(
      async ({ userId }: { userId: string }) => {
        const user = userService.findOne(userId);

        if (!user) {
          const availableIds = userService
            .findAll()
            .map((u) => u.id)
            .join(', ');

          return `用户 ID ${userId} 不存在。可用的 ID: ${availableIds}`;
        }

        return `用户信息: \n- ID: ${user.id}\n- 姓名: ${user.name}\n- 邮箱: ${user.email}\n-`;
      },
    );
  },
  name: 'query_user',
}
```

```
    description:
      '查询数据库中的用户信息。输入用户 ID，返回该用户的详细信息（姓名、邮箱、角色）。',
    schema: queryUserArgsSchema,
  },
);
},
inject: [UserService],
},
```

唯一的区别就是现在的实现用注入的 userService 来做，返回 tool

然后替换下之前的 tool：

调用的也换成这个：

再跑一下：

神光的编程秘籍

这样我们就打通了 tool 里调用 service

那自然就可以实现数据库增删改查的 tool、发送邮件的 tool

我们用 qq 邮箱的 smtp 服务发送邮件

神光的编程秘籍

拿到授权码之后，我们安装下 nodemailer

```
pnpm install nodemailer @nestjs-modules/mailer
```

在 AppModule 引入下：

```
MailerModule.forRootAsync({  
  inject: [ConfigService],  
  useFactory: (configService: ConfigService) => ({
```

```
transport: {
  host: configService.get<string>('MAIL_HOST'),
  port: Number(configService.get<string>('MAIL_PORT')),
  secure: configService.get<string>('MAIL_SECURE') === 'true',
  auth: {
    user: configService.get<string>('MAIL_USER'),
    pass: configService.get<string>('MAIL_PASS'),
  },
},
defaults: {
  from:
    configService.get<string>('MAIL_FROM')
},
}),
}),
```

这里的配置也是放在 .env 里：

```
MAIL_HOST=smtp.qq.com
MAIL_PORT=587
MAIL_SECURE=false
MAIL_USER=你的邮箱
MAIL_PASS=你的授权码
MAIL_FROM="No Reply" <你的邮箱>
```

我们把它封装成 tool

在 AiModule 加上这个 provider：

```

{
  provide: 'SEND_MAIL_TOOL',
  useFactory: (mailerService: MailerService, configService: ConfigService) => {
    const sendMailArgsSchema = z.object({
      to: z
        .email()
        .describe('收件人邮箱地址, 例如: someone@example.com'),
      subject: z.string().describe('邮件主题'),
      text: z.string().optional().describe('纯文本内容, 可选'),
      html: z.string().optional().describe('HTML 内容, 可选'),
    });

    return tool(
      async ({to, subject, text, html}: {
        to: string;
        subject: string;
        text?: string;
        html?: string;
      }) => {
        const fallbackFrom =
          configService.get<string>('MAIL_FROM')

        await mailerService.sendMail({
          to,
          subject,
          text: text ?? ' (无文本内容) ',
          html: html ?? `<p>${text ?? ' (无 HTML 内容) '}</p>`,

```

```

        from: fallbackFrom,
    });

    return `邮件已发送到 ${to}, 主题为「${subject}」`;
  },
  {
    name: 'send_mail',
    description:
      '发送电子邮件。需要提供收件人邮箱、主题，可选文本内容和 HTML 内容。',
    schema: sendMailArgsSchema,
  },
);
},
inject: [MailerService, ConfigService],
},

```

在 AiService 里注入下：

tool 调用的地方也要加一下：

这样，我们就可以用自然语言调用这个工具了：

测一下：

神光的编程秘籍

这样，邮件发送的 tool 就跑通了。

接下来实现网络搜索的 tool。

用博查的 api：

<https://open.bochaai.com/>

deepseek 的搜索就是用的这个：

挺靠谱的。

我们先搞一个 api key：

添加到 .env 文件里

```
BOCHA_API_KEY=sk-xxx
```

然后在 AiModule 添加一个 tool 的 provider:

```
{
  provide: 'WEB_SEARCH_TOOL',
  useFactory: (configService: ConfigService) => {
    const webSearchArgsSchema = z.object({
      query: z
        .string()
        .min(1)
        .describe('搜索关键词，例如：公司年报、某个事件等'),
      count: z
        .number()
        .int()
        .min(1)
        .max(20)
        .optional()
        .describe('返回的搜索结果数量，默认 10 条'),
    });

    return tool(
      async ({ query, count }: { query: string; count?: number }) => {
        const apiKey = configService.get<string>('BOCHA_API_KEY');
        if (!apiKey) {
          return 'Bocha Web Search 的 API Key 未配置（环境变量 BOCHA_API_KEY），请先在服务端配置后';
        }

        const url = 'https://api.bochaai.com/v1/web-search';
        const body = {
          query,
          freshness: 'noLimit',
          summary: true,
          count: count ?? 10,
        };
      }
    );
  }
}
```

```

const response = await fetch(url, {
  method: 'POST',
  headers: {
    Authorization: `Bearer ${apiKey}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify(body),
});

if (!response.ok) {
  const errorText = await response.text();
  return `搜索 API 请求失败, 状态码: ${response.status}, 错误信息: ${errorText}`;
}

let json: any;
try {
  json = await response.json();
} catch (e) {
  return `搜索 API 请求失败, 原因是: 搜索结果解析失败 ${(e as Error).message}`;
}

try {
  if (json.code !== 200 || !json.data) {
    return `搜索 API 请求失败, 原因是: ${json.msg ?? '未知错误'}`;
  }

  const webpages = json.data.webPages?.value ?? [];
  if (!webpages.length) {
    return '未找到相关结果。';
  }

  const formatted = webpages
    .map(
      (page: any, idx: number) =>
        `引用: ${idx + 1}
标题: ${page.name}
URL: ${page.url}
摘要: ${page.summary}
网站名称: ${page.siteName}
网站图标: ${page.siteIcon}
发布时间: ${page.dateLastCrawled}`
    )
    .join('\n\n');

  return formatted;
} catch (e) {

```

```
        return `搜索 API 请求失败, 原因是: 搜索结果解析失败 ${e as Error}.message`;
    }
},
{
    name: 'web_search',
    description:
        '使用 Bocha Web Search API 搜索互联网网页。输入为搜索关键词 (可选 count 指定结果数量), 返
    schema: webSearchArgsSchema,
},
);
},
inject: [ConfigService],
},
```

就是从配置文件拿到 apikey, 通过 http 调用搜索接口, 把结果格式化后给大模型。

然后在 AiService 里用一下:

然后来测一下

当然，sse 还是用界面测更好，我们加一个 html

public/ai-sse-test.html

```
<!doctype html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <title>AI SSE Chat 测试</title>
  <style>
    * {
      box-sizing: border-box;
    }

    body {
      margin: 0;
      min-height: 100vh;
      font-family: system-ui, -apple-system, BlinkMacSystemFont, 'SF Pro Text',
        'Segoe UI', sans-serif;
      display: flex;
      align-items: center;
      justify-content: center;
      background: #f5f5f5;
      padding: 24px 16px;
    }

    .shell {
      width: 100%;
```

```
    max-width: 720px;
  }

  h1 {
    font-size: 20px;
    margin: 0 12px;
  }

  .card {
    background: #ffffff;
    border-radius: 10px;
    border: 1px solid #e5e7eb;
    box-shadow: 0 4px 12px rgba(15, 23, 42, 0.06);
    padding: 16px 18px 18px;
  }

  label {
    display: block;
    font-size: 13px;
    margin-bottom: 6px;
    color: #6b7280;
  }

  textarea {
    width: 100%;
    min-height: 80px;
    padding: 8px 10px;
    border-radius: 8px;
    border: 1px solid #d1d5db;
    resize: vertical;
    font-family: inherit;
    font-size: 14px;
    outline: none;
  }

  textarea::placeholder {
    color: #9ca3af;
  }

  textarea:focus {
    border-color: #3b82f6;
    box-shadow: 0 0 1px rgba(59, 130, 246, 0.3);
  }

  .controls {
    display: flex;
```

```
        align-items: center;
        margin-top: 10px;
        gap: 10px;
    }

    button {
        padding: 6px14px;
        border-radius: 999px;
        border: 1px solid #2563eb;
        background: #3b82f6;
        color: #ffffff;
        font-size: 13px;
        cursor: pointer;
    }

    button:disabled {
        opacity: 0.7;
        cursor: not-allowed;
    }

    .status {
        font-size: 12px;
        color: #6b7280;
    }

    .output {
        margin-top: 16px;
        padding: 10px10px;
        border-radius: 8px;
        background: #111827;
        color: #e5e7eb;
        font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas,
            'Liberation Mono', 'Courier New', monospace;
        white-space: pre-wrap;
        max-height: 360px;
        overflow-y: auto;
    }
</style>
</head>
<body>
    <div class="shell">
        <div class="card">
            <h1>AI SSE Chat 测试</h1>
            <label for="query">输入你的问题: </label>
            <textarea
                id="query">
```

```

        placeholder="请输入要发送给 AI 的问题..."
    </textarea>
    <div class="controls">
        <button id="sendBtn">开始对话 (SSE) </button>
        <div class="status" id="status">状态: 待机</div>
    </div>
    <div class="output" id="output"></div>
</div>

<script>
    const sendBtn = document.getElementById('sendBtn');
    const queryInput = document.getElementById('query');
    const outputEl = document.getElementById('output');
    const statusEl = document.getElementById('status');

    let es = null;

    function closeEventSource() {
        if (es) {
            es.close();
            es = null;
        }

        sendBtn.disabled = false;
    }

    sendBtn.onclick = () => {
        const query = queryInput.value.trim();
        if (!query) {
            alert('请输入问题');
            return;
        }

        closeEventSource();
        outputEl.textContent = '';
        sendBtn.disabled = true;
        statusEl.textContent = '状态: 连接中...';

        const url = `/ai/chat/stream?query=${encodeURIComponent(query)}`;
        es = new EventSource(url);

        es.onopen = () => {
            statusEl.textContent = '状态: 已连接, 流式接收中...';
        };
    };

```



```
    es.onmessage = (event) => {  
      // 后端每个 chunk 用 data 发过来  
      outputEl.textContent += event.data;  
    };  
  
    es.onerror = () => {  
      statusEl.textContent = '状态: 连接结束或发生错误';  
      closeEventSource();  
    };  
  };  
};  
  
window.addEventListener('beforeunload', closeEventSource);  
</script>  
</body>  
</html>
```

同样是 ai 写的页面，主要是用 EventSource 对接 sse 接口。

在 AppModule 加一下静态文件的访问

安装用到的包

```
pnpm install @nestjs/serve-static
```

跑一下：

网络搜索和发送邮件的 tool 都跑通了。

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

我们梳理了豆包、OpenClaw 定时任务的实现思路。

创建了 Nest 后端项目，基于 LangChain + tool 实现了工具调用。

在 service 里加上了 Agent Loop，并且用 stream 方法实现了流式，提供 sse 接口。

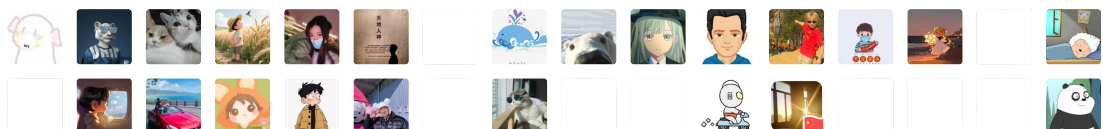
然后我们把 tool 封装到 provider 实现了 tool 里调用 service。

之后分别封装了邮件发送 tool、网络搜索 tool。

综合测试了下，可以通过自然语言调用这些 tool。

下篇我们继续来实现数据库增删改查的 tool、定时任务的 tool，然后实现完整定时任务机制。

2774 人付费



上一篇

Nest + LangChain 实现基于 SSE 的流式 ai
接口

下一篇

Nest + tool 实现 OpenClaw 同款定时任务功
能（下）
